

Results 1 X	
explain analyze select oi.order_id, p.product_name Enter a SQL expression to filter results (use Ctrl+Space)	
Grid	A-Z QUERY PLAN
1	Hash Join (cost=27.09..41.32 rows=7 width=274) (actual time=0.058..0.066 rows=4.00 loops=1)
2	Hash Cond: (p.product_id = oi.product_id)
3	Buffers: shared hit=2 dirtied=1
4	-> Seq Scan on products p (cost=0.00..13.00 rows=300 width=222) (actual time=0.014..0.015 rows=5.00 loops=1)
5	Buffers: shared hit=1 dirtied=1
6	-> Hash (cost=27.00..27.00 rows=7 width=28) (actual time=0.033..0.035 rows=4.00 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared hit=1
9	-> Seq Scan on order_items oi (cost=0.00..27.00 rows=7 width=28) (actual time=0.007..0.009 rows=4.00 loops=1)
10	Filter: (order_id = 1)
11	Rows Removed by Filter: 3
12	Buffers: shared hit=1
13	Planning:
14	Buffers: shared hit=9
15	Planning Time: 0.151 ms
16	Execution Time: 0.099 ms

PostgreSQL executes this query using a **Hash Join** to combine data from both tables. First, it performs a **Sequential Scan** (Seq Scan) on the *order_items* table, filtering out the rows to keep only those where *order_id* = 1, and builds a temporary hash table in memory from the results. Next, it reads the *products* table via another **Sequential Scan** and scans it against the built hash table to match rows where *product_id* matches. Sequential scans were chosen by the query planner instead of index scans because both tables currently contain a very small number of rows, making a full table scan faster.